



Data Mining & Unsupervised Learning

লেবেল ছাড়া **Data** থেকে লুকানো **Pattern** বের করা

১. Data Mining কী — আর KDD Pipeline কী?

প্রথমে বুঝো: **Data Mining vs KDD**

অনেকে মনে করে Data Mining মানেই সব — কিন্তু আসলে Data Mining হলো একটা বড় **process** এর একটা ধাপ।

সেই বড় process টার নাম হলো **KDD — Knowledge Discovery in Databases**।

KDD Pipeline — ধাপে ধাপে:

Raw Data



Pre-Processing ← ময়লা পরিষ্কার করা (missing values, duplicates)



Transformation ← Data কে কাজের format এ আনা (scaling, encoding)



Data Mining ← আসল pattern খোঁজার কাজ (এটাই মূল ধাপ)



Evaluation/Insight ← যা পেলাম সেটা কি কাজের? মানুষকে বোঝানো

Data Mining এর Goal:

Large dataset থেকে এমন insight বের করা যা **non-trivial** (সহজে চোখে পড়ে না), **valid** (সত্যি), এবং **actionable** (কাজে লাগানো যায়)।

২. Association Rule Mining — "যে কেনে X, সে কেনে Y"

Market Basket Analysis কী?

Supermarket এর কথা ভাবো। লক্ষ লক্ষ customer প্রতিদিন কেনাকাটা করছে। তুমি জানতে চাও — কোন জিনিসগুলো মানুষ একসাথে কেনে?

এটাই **Market Basket Analysis** — একসাথে frequently কেনা item এর combination খোঁজা।

Rule এর format:

$X \rightarrow Y$

"যদি কেউ X কেনে, তাহলে সে Y-ও কিনবে"

উদাহরণ:

Transaction ID	কী কিনল
1	Milk, Bread, Diapers
2	Butter, Diapers, Beer
3	Milk, Diapers, Beer
4	Bread, Butter

প্রশ্ন: **Diapers** $\rightarrow$ **Beer**? মানে যে Diapers কেনে, সে কি Beer ও কেনে?

Transaction 2 আর 3 তে দুটোই আছে — মনে হচ্ছে একটা সম্পর্ক আছে!

Real life এ এটা কোথায় কাজে লাগে:

- **Cross-selling:** "আপনি কি এটাও দেখবেন?" (Amazon এর "Customers also bought")
- **Store layout:** Diapers এর পাশে Beer রাখো $\rightarrow$ বিক্রি বাড়বে
- **Recommendation engine:** Netflix, Spotify এর suggestion

৩. Rule Strength মাপার তিনটা Metric

কোনো rule কতটা শক্তিশালী সেটা তিনটা জিনিস দিয়ে মাপা হয়:



Metric ১: Support

Definition: পুরো dataset এ এই item combination কতবার দেখা গেছে?

$\text{Support}(X \rightarrow Y) = P(X \cap Y) = (\text{X আর Y একসাথে আছে এমন transactions}) / (\text{মোট transactions})$

উদাহরণ (**Diapers** $\rightarrow$ **Beer**):

- মোট transaction: 4টা
- Diapers এবং Beer দুটোই আছে: Transaction 2, 3 $\rightarrow$ ২টা

$\text{Support} = 2/4 = 0.5$ (50%)

Support কেন দরকার? — Rare item filter করতে। যদি কোনো rule মাত্র ১-২ বার দেখা গেছে, সেটা কাকতালীয় হতে পারে।



Metric ২: Confidence

Definition: X যখনই কেনা হয়েছে, তার মধ্যে কতবার Y-ও কেনা হয়েছে?

$$\text{Confidence}(X \rightarrow Y) = P(Y|X) = P(X \cap Y) / P(X)$$

উদাহরণ (**Diapers** → **Beer**):

- Diapers আছে এমন transaction: 1, 2, 3 → ৩টা
- Diapers AND Beer আছে: 2, 3 → ২টা

$$\text{Confidence} = 2/3 = 0.67 \text{ (67\%)}$$

মানে: যখনই কেউ Diapers কিনেছে, ৬৭% ক্ষেত্রে Beer-ও কিনেছে।

কিন্তু **Confidence** একাই যথেষ্ট না! কারণ Beer হয়তো এমনিতেই সবাই কেনে — তাহলে এই rule টা special কিছু না।



Metric ৩: Lift

Definition: X আর Y এর সম্পর্ক কি সত্যিই আছে, নাকি এটা coincidence?

$$\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / P(Y)$$

মানে: "আমরা যা দেখলাম" vs "যদি দুটো independent হতো তাহলে কী হতো"

উদাহরণ:

- Beer এর overall probability: Transaction 2, 3 তে Beer আছে → $P(\text{Beer}) = 2/4 = 0.5$
- $\text{Confidence}(\text{Diapers} \rightarrow \text{Beer}) = 0.67$

$$\text{Lift} = 0.67 / 0.5 = 1.34$$

Lift এর মানে বোঝো:

Lift

মানে কী

> 1 X আর Y এর মধ্যে **positive** সম্পর্ক আছে — rule কাজের

$= 1$ X আর Y **independent** — কোনো সম্পর্ক নেই

< 1 X আর Y এর মধ্যে **negative** সম্পর্ক — X কিনলে Y কম কেনে

আমাদের Lift = 1.34 > 1 → Diapers → Beer rule টা **valid!**

8. Unsupervised Learning — Label ছাড়া শেখা

Supervised vs Unsupervised

	Supervised	Unsupervised
Data	X আর y দুটোই আছে	শুধু X আছে, y নেই
লক্ষ্য	y predict করা	নিজেই structure খুঁজে বের করা
উদাহরণ	Spam detection	Customer grouping

সহজ উদাহরণ: Supervised = শিক্ষক বলে দিচ্ছেন "এটা A, এটা B" Unsupervised = শিক্ষক নেই, তুমি নিজেই মিল-অমিল দেখে group করছ

Unsupervised Learning এ তিনটা প্রধান কাজ:

১. **Clustering** → Similar data point গুলো একসাথে group করা

- উদাহরণ: Customer দেব ভাগ করা — "বড়লোক customer", "মধ্যবিত্ত customer", "কম খরচের customer"

২. **Dimensionality Reduction** → অনেক features কে কমিয়ে কম features এ আনা

- উদাহরণ: ১০০টা feature থেকে মূল ২-৩টা feature বের করা

৩. **Anomaly Detection** → যা normal না তাকে খুঁজে বের করা

- উদাহরণ: Credit card fraud — এই transaction টা unusual কেন?

সবচেয়ে বড় চ্যালেঞ্জ:

কোনো "সঠিক উত্তর" নেই যার সাথে তুলনা করবে।

Supervised Learning এ বলতে পারো "model ৯৫% accurate" — কারণ সঠিক উত্তর জানা আছে।
Unsupervised এ কীভাবে বুঝবে clustering "ভালো" হয়েছে? এটা **subjective** এবং mathematical heuristics দিয়ে মাপতে হয়।

৫. K-Means Clustering — সবচেয়ে বিখ্যাত Clustering Algorithm

মূল লক্ষ্য:

N টা data point কে K টা group (cluster) এ ভাগ করো, যাতে প্রতিটা **cluster** এর ভেতরে **data point** গুলো একে অপরের কাছাকাছি থাকে।

এটা measure করা হয় **WCSS (Within-Cluster Sum of Squares)** দিয়ে:

প্রতিটা point আর তার cluster center এর মধ্যে দূরত্বের বর্গের যোগফল। এটা যত কম, clustering তত ভালো।

K-Means কীভাবে কাজ করে — ধাপে ধাপে:

ধরো তোমার কাছে **customer data** আছে এবং তুমি ৩টা **group** বানাতে চাও (**K=3**):

ধাপ ১: **Initialize**

Randomly ৩টা জায়গায় "কাল্পনিক center" (centroid) রাখো

Data space তে randomly: C1, C2, C3 বসেও

ধাপ ২: **Assign**

প্রতিটা data point দেখো — সে কোন centroid এর সবচেয়ে কাছে? সেই cluster এ assign করো।

Euclidean Distance = $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

সবচেয়ে কম distance → সেই cluster এ যাও

ধাপ ৩: **Update**

এখন প্রতিটা cluster এ যে points আছে, তাদের গড় (**mean**) বের করো। সেটাই নতুন centroid।

নতুন C1 = cluster 1 এর সব point এর average position

ধাপ ৪: **Repeat**

ধাপ ২ আর ৩ বারবার করো, যতক্ষণ centroid আর নড়ছে না (convergence)।

Visualization:

শুরুতে: কয়েক iteration পরে: শেষে:

$\times \times \circ \circ$ $\times \times \mid \circ \circ$ [$\times$ গ্রুপ] [$\circ$ গ্রুপ] [Δ গ্রুপ]
 $\times \Delta \circ \Delta$ $\times \Delta \mid \circ \Delta$
 $\Delta \Delta \times \circ$ —————|—————

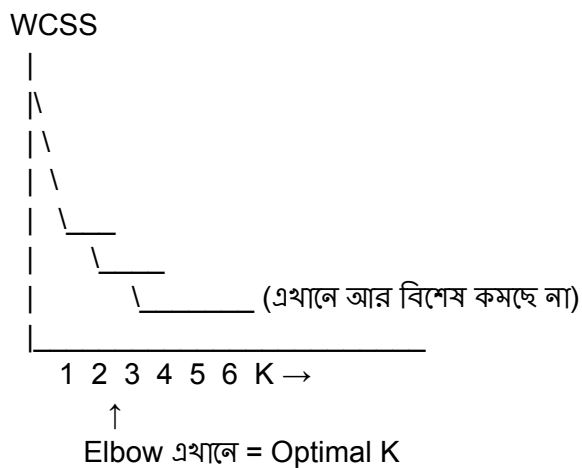
গুরুত্বপূর্ণ সীমাবদ্ধতা:

K-Means চালানোর আগেই তোমাকে **K** বলে দিতে হবে — algorithm নিজে থেকে বুঝবে না কতটা group বানাতে হবে। তাহলে সঠিক K কীভাবে বেছে নেবে? এখানেই আসে Elbow Method।

৬. Elbow Method — সঠিক K খোঁজা

পদ্ধতি:

K = 1, 2, 3, 4, ... 10 সব দিয়ে K-Means চালাও। প্রতিটার জন্য WCSS বের করো। Graph আঁকো।



Elbow বোঝার Logic:

K বাড়ালে WCSS কমবেই — কারণ বেশি cluster মানে প্রতিটা point তার center এর আরও কাছে।

কিন্তু একটা জায়গায় এসে আর বিশেষ কমছে না — এটাই **Elbow point** বা **Diminishing Returns** এর শুরু।

এই point এর পরে K বাড়ানো মানে শুধু complexity বাড়ানো, কোনো real benefit নেই।

৭. PCA — অনেক Feature কে কমিয়ে আনা

সমস্যা: **Curse of Dimensionality**

ধরো তোমার কাছে ১০০টা feature আছে। তাহলে:

- Model train করতে অনেক সময় লাগে
- ১০০ dimension এ visualize করা impossible
- অনেক feature আসলে একে অপরের সাথে correlated — redundant

PCA কী করে?

PCA = Principal Component Analysis

অনেক correlated features কে নিয়ে সেগুলোকে কম সংখ্যক **uncorrelated** নতুন features এ রূপান্তর করে, কিন্তু information যতটা সম্ভব ধরে রাখে।

Principal Component বোঝো:

PC1 (First Principal Component):

Data এর মধ্যে যে direction এ সবচেয়ে বেশি variation (variance) আছে, সেই direction।

সহজ ভাষায়: data কে কোন দিক থেকে দেখলে সবচেয়ে বেশি spread দেখা যায়?

PC2 (Second Principal Component):

PC1 এর সাথে **perpendicular (90°)** — এবং বাকি যে variance আছে সেটা capture করে।

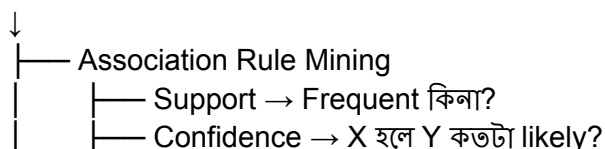
Original:	PCA এর পরে:
Height	PC1 (মূল variation)
Weight	→ ↓
Age	PC2 (বাকি variation)
Income	
BMI	
(৫টা feature)	(২টা feature, তবু বেশিরভাগ info আছে)

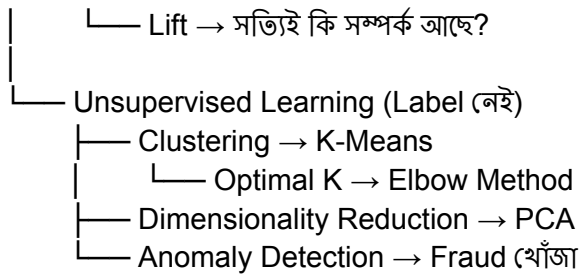
কেন PCA দরকার?

- **Speed:** কম feature → দ্রুত training
- **Visualization:** ১০০ dimension কে ২D বা ৩D তে দেখানো যায়
- **Noise** কমানো: অপ্রয়োজনীয় features বাদ যায়

পুরো Chapter এর সম্পর্ক

Data Mining (KDD এর একটা ধাপ)





এখন **Genetic Algorithm** আর **A* Search** এর content দাও — ওগুলো একইভাবে বিস্তারিত করে দিই! তুমি বললে এই দুটো তোমার জন্য সবচেয়ে important 🎯